Full-Stack Developer Roadmap

3 YOE → Senior Full-Stack Engineer

React / Angular | Node.js | PostgreSQL | Redis | WebSockets | SSE | System Design | DevOps

18-Month Project-Based Learning Plan

5	8	18
Portfolio Projects	Learning Phases	Months

Each phase builds on the last. Topics become features in real portfolio projects.

Ship code you can demo in every interview.

About This Roadmap

This is an 18-month, project-based learning plan designed to take you from 3 years of frontend experience to a well-rounded senior full-stack engineer. Every phase combines concrete topics to learn with a real project that implements those topics as features — so you always have working code to show, not just notes. By the end, you will have shipped 5 portfolio projects, a published npm library, a live SaaS product, and the system design knowledge to confidently interview for senior and full-stack roles.

5 Projects	8 Phases	18 Months	1 Live SaaS

1 Senior Frontend Mastery

Deep-dive Angular/React + modern tooling

Months 1–3

TOPICS TO LEARN

React 19 patterns · Server components · Angular signals · Micro-frontends · Zustand / NgRx · Code splitting · Lazy loading · Bundle analysis · Tree shaking · CSR / SSR / ISR / SSG · Accessibility (WCAG 2.2) · Web Vitals (LCP/CLS/INP) · Testing (Vitest, Cypress)

PROJECT / IMPLEMENTATION

Project 1: DevFlow Dashboard — Portfolio, Resume

A GitHub-like developer dashboard displaying repos, activity, and streaks. Built with Next.js (App Router) + Angular micro-frontend widget.

- → Server components + client islands for hybrid rendering
- → Real-time GitHub stats via Server-Sent Events (SSE)
- → Full Lighthouse 100 score — measure and fix every Web Vital
- → Keyboard-navigable, WCAG 2.2 AA compliant throughout
- → Vitest unit tests + Cypress E2E coverage

2 Design Systems & UI Engineering

Component libraries, tokens, Storybook

Months 3–5

TOPICS TO LEARN

Design tokens · Storybook 8 · Radix / shadcn/ui · Compound components · Polymorphic components · CSS custom properties · Tailwind v4 · Figma tokens plugin · Visual regression tests · Theme switching · Animation (Framer Motion) · Chromatic CI

PROJECT / IMPLEMENTATION

Project 2: Forge UI — Component Library — Portfolio, Resume

A publishable npm component library with 20+ components, design token system, dark mode, and Storybook docs. Stands out heavily on resumes.

- → Design token pipeline: Figma → JSON → CSS vars → Tailwind
- → Polymorphic Button, Card, Modal, Form, DataTable components
- → Storybook with interactive docs, a11y addon, viewport controls
- → Automated visual regression via Chromatic on every PR
- → Published to npm with semantic versioning + changelog

3 Frontend Performance Engineering

Perf profiling, memory, rendering pipeline

Months 5–6

TOPICS TO LEARN

- Chrome DevTools deep dive
- Rendering pipeline
- Memory profiling
- Layout thrashing
- Virtualization (TanStack)
- Image optimization
- Font subsetting
- HTTP/2 + HTTP/3
- Resource hints (preload/prefetch)
- Service workers
- Web Workers
- WASM basics
- RUM (Sentry/Datadog)

PROJECT / IMPLEMENTATION

Add-on to Project 1: Performance Audit Report — Resume

Take the DevFlow Dashboard and ship a documented performance transformation — before/after metrics you can show in interviews.

- → TanStack Virtual for activity feed — 10,000 rows smooth scroll
- → Web Worker for heavy data processing (GitHub stats aggregation)
- → Service worker + cache-first strategy for offline support
- → Document LCP improvement from baseline (target: under 1.5s)
- → Write a technical blog post about the optimisation journey

4 Backend Foundations (Node.js)

APIs, databases, auth, security

Months 6–9

TOPICS TO LEARN

- Node.js / Bun runtime
- REST API design
- Hono / Fastify / Express
- PostgreSQL
- Drizzle ORM / Prisma
- Redis
- JWT + refresh tokens
- OAuth 2.0 / OIDC
- Session management
- Bcrypt / Argon2
- CORS, CSRF, XSS
- Input validation (Zod)
- Rate limiting
- API versioning
- Swagger / OpenAPI

PROJECT / IMPLEMENTATION

Project 3: Collab — Real-time Workspace — Portfolio, Resume

A Notion-like collaborative workspace. Ship the full backend: REST + WebSocket + auth + database — then connect your frontend from Phases 1 and 2.

- → Auth: JWT + refresh token rotation, Google OAuth, magic links
- → PostgreSQL schema with Drizzle ORM + proper migrations
- → WebSockets for real-time collaborative document editing
- → Redis for session cache + pub/sub across instances
- → Rate limiting middleware per user, per IP address
- → OpenAPI spec with Swagger UI, auto-generated from Zod schemas
- → Full OWASP Top 10 audit checklist in README

5 Real-time, Sockets & Streaming

WebSockets, SSE, queues

Months 9–10

TOPICS TO LEARN

- WebSocket protocol
- Socket.io
- Server-Sent Events
- Long polling (legacy)
- Message queues (BullMQ)
- Event-driven architecture
- Presence systems
- Operational transforms (OT)
- CRDT basics
- Reconnect strategies

PROJECT / IMPLEMENTATION

Add-on to Project 3: Live Features Sprint

Resume

Extend Collab with production-grade real-time features that demonstrate deep understanding of async systems.

- → Presence indicators — who is online, live cursors on documents
- → SSE-based notification system (no polling overhead)
- → BullMQ background jobs: email, exports, AI processing pipeline
- → Graceful reconnection with exponential backoff on the client

6 DevOps, Infra & Production Deployment

Docker, CI/CD, cloud, monitoring

Months 10–12

TOPICS TO LEARN

Docker + compose | GitHub Actions CI/CD | AWS / GCP / Railway | Nginx / reverse proxy | SSL/TLS (Let's Encrypt) | Kubernetes basics | Environment secrets | Sentry error tracking | Grafana + Prometheus | Log aggregation | Zero-downtime deploys | Database backups | CDN (Cloudflare)

PROJECT / IMPLEMENTATION

Ship Collab to Production

Portfolio Resume

Take the full Collab app and ship it with a production-grade infrastructure setup — something you can demo live in any interview.

- → Dockerize frontend + backend + worker + Redis + Postgres
- → GitHub Actions: lint → test → build → deploy pipeline
- → Deploy to Railway / Render / AWS with custom domain + SSL
- → Grafana dashboard: requests/sec, error rate, p95 latency
- → Runbook + architecture diagram in repo wiki

7 System Design & Scalability

Caching, scaling, architecture patterns

Months 12–15

TOPICS TO LEARN

CAP theorem | Horizontal scaling | Load balancing | DB replication | Read replicas | Sharding | Caching strategies | Cache invalidation | CDN strategy | API Gateway | Circuit breakers | Idempotency | gRPC basics | Message brokers (Kafka)

PROJECT / IMPLEMENTATION

Project 4: Scale Collab to 10k Users

Resume

Design and partially implement how Collab would scale. The architecture document alone is a strong portfolio piece for senior roles.

- → Multi-layer caching: Redis L1 (hot data) + CDN L2 (static assets)
- → DB read replica for analytics queries, reducing write DB load
- → Distributed rate limiting with Redis sliding window algorithm
- → Write system design doc: How to scale Collab to 100k users
- → Load test with k6: measure break points, document results

★ Capstone: Ship a SaaS from Scratch

Your magnum opus — every skill integrated

Months 15–18

PROJECT / IMPLEMENTATION

Project 5: PulseMetrics — Analytics SaaS

Portfolio Resume

A lightweight Plausible/PostHog alternative. Every skill from every phase ships together into one production product with real users.

- → Design system: custom component library from Phase 2
- → Auth: email/password + Google OAuth + org/team multi-tenancy
- → Billing: Stripe subscriptions, webhooks, usage metering
- → Event ingestion: high-throughput endpoint + Kafka queue
- → Real-time dashboard: WebSocket live visitor counts
- → SSE alerts: notify when traffic spikes are detected
- → Redis caching for dashboard queries (cache-aside pattern)
- → Rate limiting per API key + per org subscription tier
- → Full CI/CD: test → build → deploy → smoke test
- → Monitoring: Sentry + Grafana + uptime checks
- → Public landing page (Next.js) with Lighthouse 100 score
- → Write-up: architecture decisions + trade-offs document

Study Alongside Every Phase

These resources run in parallel with your project work — 30 minutes a day compounds dramatically over 18 months.

Books

→ Alex Xu — System Design Interview Vol 1 & 2

→ Kyle Simpson — You Don't Know JS (YDKJS) series

→ Addy Osmani — Learning Patterns (free online)

→ Designing Data-Intensive Applications — Martin Kleppmann

Practice

→ Neetcode 150 — DSA for technical interviews

→ roadmap.sh/frontend + roadmap.sh/backend

→ Frontend Masters — Advanced courses

→ Build in public — tweet your progress weekly

Visibility

→ Write technical articles on dev.to or personal blog

→ Contribute to open-source (good first issues)

→ Record Loom walkthroughs of each project

→ LinkedIn posts about what you shipped each month

Interview Strategy & Career Tips

Resume

→ List the 5 projects with impact metrics: 'Reduced LCP from 4.2s to 0.9s', 'Published npm package with 500+ downloads'

→ Headline skill: Full-Stack Engineer (React / Node.js / PostgreSQL)

→ Add a GitHub profile link — every project should have a clean README with live demo link

Interviews

→ For system design: use the Collab scaling doc as a template — you have already solved this problem once

→ For coding rounds: Neetcode 150 covers 90% of patterns. Focus on arrays, trees, graphs, dynamic programming

→ For frontend rounds: be ready to implement a debounce, a virtual list, or a state machine from scratch

Job Search

→ Target companies that list both frontend and full-stack roles — they value frontend depth + backend breadth

→ Your PulseMetrics SaaS is a conversation starter: 'I built and shipped this — here is the architecture'

→ Aim for: Mid-level Full-Stack, Senior Frontend, or Full-Stack Engineer titles at 8-15 person eng teams

Ship code. Write about it. Get hired.

Every week you are building — not just studying. That is what separates you from the crowd.